FOODLOOP

Safe food-waste collection, routing and proof.

Features, operating workflow, product approach and technology stack

A PPT-ready project briefing

Prepared October 2026 | Based on the current application code

PROJECT PURPOSE

FoodLoop helps businesses send expired, spoiled or damaged food waste to an appropriate downstream route, coordinate consolidated pickups, and retain a verifiable disposal trail.

What this document is

A grounded description of features visible in the current implementation, with assumptions and pre-launch work called out separately. It is intended as source material for a presentation, not as a claim of regulatory certification or production readiness.

01 / PRODUCT

FoodLoop at a glance

FoodLoop is a mobile-first web application delivered as an installable Progressive Web App (PWA). Its core is a managed B2B disposal workflow: businesses record waste lots, the platform applies routing rules, pickup operators consolidate and execute runs, and completed runs produce a record that can be independently checked for post-issue changes.

Product goals

- Keep expired or spoiled food out of human-food and animal-feed pathways; route it only to certified recovery facilities.
- Avoid inefficient single-business trips by grouping compatible lots into facility-specific runs.
- Give businesses pickup progress, measured weights, facility details, invoices and a disposal record.
- Give administrators control of business verification, facilities, collector invites, users and pricing.

Primary users

Role	What they do	Typical outcome
Business owner	Registers an organization, adds lots, checks collection progress and records, and manages billing.	A planned pickup and documented disposal of eligible waste.
Collector / hub operator	Sets vehicle and availability, accepts or receives runs, schedules collection, records weights and closes a run.	A route-sized run with a frozen fare calculation.
Platform administrator	Verifies businesses, maintains facility and certificate details, invites collectors, manages users and plans.	A governed operating network and platform-wide oversight.

Product boundary

The current product is not a food-rescue marketplace. In-date, fresh food is refused by the classification rules; there is no resale or donation route in the current rule engine.

02 / RULE ENGINE

Routing policy and item categories

Each lot is evaluated using its category, condition and (for date-bearing categories) expiry date. The result is a route plus a reason, urgency, days remaining and routes that are locked. The frontend shows user-facing route letters A-D; the database retains stable internal route keys to preserve previously issued data.

User-facing route	Purpose	Rule summary
A - Process	Pulp, jam, pickle and dehydration units.	Overripe but usable food can go to processing.
B - Feed	Animal-feed makers.	Only non-expired, feed-suitable damaged food can use feed.
C - Recover	Certified compost or anaerobic digestion.	Expired, spoiled, or non-feed-suitable damaged food is directed here; process and feed are locked.
D - Packaging	Packaging recycling.	Empty packaging has a fixed packaging route.

Current catalog

Vegetables; Fruits; Dairy; Bakery and bread; Grains, flour and pulses; Packaged snacks and biscuits; Cooked and ready-to-eat food; Mixed kitchen waste; and Empty packaging. The former product-specific keys (for example tomato, banana and leafy greens) resolve to the broader vegetables or fruits category so older lots remain readable.

Decision sequence

- Fixed categories (mixed kitchen waste and empty packaging) take their designated routes.
- If a lot is expired, lock process and feed and use certified recovery only.
- If spoiled, use recovery; if bruised, allow feed only when the category is marked feed-suitable.
- If overripe but usable, send to processing; fresh, good-condition in-date food is rejected as not waste.

Important caveat

These are application decision rules, not a food-safety inspection or legal determination. Confirm rules and facility eligibility with qualified operators and applicable FSSAI and local requirements before a real-world launch.

03 / OPERATING WORKFLOW

End-to-end workflow | intake to pickup plan

Step	Actor / action	System behavior
1. Join	Business signs up; collector uses an admin-issued invite.	Password-authenticated account. Business organization starts pending verification; collector invite is one-use and time-limited.
2. Verify	Administrator reviews and verifies or declines the business.	Organization verification is tracked. Unverified business lots are excluded from automatic pickup planning.
3. Add lot	Business chooses a food category and condition, enters kilograms and any required expiry date.	Input validation prevents invalid quantity, missing required date and fresh, good-condition waste submissions.
4. Classify	Rules derive the destination route and explain the decision.	Preview returns the route, reason, urgency and locked routes before the lot is saved.
5. Match	Business sees a suitable downstream facility.	Active route-compatible facilities are ranked using distance, capacity and configured rate; certification is required for recovery/recycling routes.
6. Pool	Eligible lots wait until a useful facility group exists.	Default minimum pickup threshold is 10 kg. Too-small groups remain waiting; a configured vehicle capacity can split a large group into runs.

Experience principle

The business sees its own lot and pickup detail, including destination and certificate information, without exposure to other businesses' route/contact details or facility commercial scoring/prices.

04 / OPERATING WORKFLOW

End-to-end workflow | dispatch to evidence

Step	Actor / action	System behavior
7. Dispatch	Collector receives or claims a run.	Available active collectors with enough vehicle capacity are ranked by proximity to the first stop, with a load penalty to spread work. If no one fits, the run remains open.
8. Route and time	Collector reviews stops and confirms an arrival time.	Stops are ordered with a nearest-neighbor estimate. Businesses receive in-app notifications and see the collector/time. The time window is two hours.
9. Collect	Collector marks a planned run as collected.	Run state moves from planned to collected; businesses are notified.
10. Weigh and close	Collector enters hub and receiving-facility weights.	A mismatch is flagged when either comparison is beyond the configured 5% tolerance. Closing triggers the disposal record, business billing and fare settlement.
11. Verify	Business, admin or public verifier checks a record code/QR.	The verifier recomputes the record hash and checks its link to the previous record. A PDF contains facility, weights, lot lines, QR and hashes.

Status progression

Waiting lot -> Planned run -> Collected -> Closed. A closed run includes expected, hub and facility weights, mismatch status, a record code, record PDF, business invoice computation, and the collector fare frozen at closure.

Tamper-evident, not blockchain

A record is SHA-256 chained to the previous record. Editing the payload or breaking its link is detectable by verification. The system is not a blockchain and the PDF explicitly says it is not an official FSSAI certificate.

05 / FEATURE INVENTORY

Feature map by role

Area	Implemented capabilities
Business	Mobile-first onboarding and sign-in; business verification status; add/preview/delete waiting lots; lot history and route explanation; pickup status, schedule and collector contacts; facility map/certificate details; record verification and PDF; CSV reports; notifications; profile/password/account controls; billing plans and invoices.
Collector	Invite-based joining; vehicle capacity, base location and availability profile; eligible/open run list; assignment and decline/reassignment; route map and stop details; pickup scheduling; collected/closed state actions; hub/facility weigh-in; fare estimate and closed fare; in-app notifications.
Administrator	Dashboard KPIs; verify/reject businesses; create and manage facilities and route eligibility; record certification number and upload/view certificate document; invite collectors; enable/disable users and issue password reset codes; manage plans/rates; inspect billing/revenue and record lists.
Shared/public	Public record-code verification; generated downloadable/inline disposal PDF with QR; facility certificate viewing for signed-in users; legal/privacy information; responsive PWA shell.

User-facing records and estimates

The interface shows collected kilograms, route/facility details, status, confirmed collection timing, in-app updates, record counts, invoices and approximate environmental impact. CO2e and meal-equivalent figures are explicitly placeholders in configuration and must not be presented as audited outcomes.

06 / IMPLEMENTATION

Technology stack and architecture

Layer	Technology	Responsibility
Client	HTML5, CSS3, vanilla JavaScript	Single-page, hash-routed application; mobile-first views; role-gated screens; API requests; PWA install metadata.
Web/API	Python 3.12; FastAPI; Uvicorn	REST endpoints, request validation, route handlers, lifecycle/startup and HTTP headers.
Persistence	SQLAlchemy 2; SQLite by default; PostgreSQL supported	Users, organizations, lots, facilities, pickups, assignments, invoices, subscriptions, notifications and records.
PDF / QR	ReportLab; Segno	Downloadable disposal evidence PDF and QR link to public record verification.
Client libraries	Leaflet; Chart.js; bundled HTML5 QR library	Maps and dashboard charts are integrated; an HTML5 QR package is bundled in static assets.
Payments	Razorpay integration (optional); development simulator	Test-key checkout and server-side signature verification; simulated payment only outside production when no gateway keys are configured.
Operations	Docker; Docker Compose; Postgres; persistent upload volume	Containerized deployment path. The app is served on port 8000 and should sit behind HTTPS in production.

Request and data path

Mobile browser / installed PWA -> same-origin FastAPI endpoints -> domain services (classification, matching, dispatch, billing, records) -> SQLAlchemy database. Uploaded facility certificate documents are stored under the configured upload directory. The service worker caches the app shell/static resources, but API requests remain network-only; the application does not support offline lot creation or offline transactions.

Local vs deployed

SQLite is the default local database. Docker Compose wires the web service to PostgreSQL and a persistent upload volume. Real users require secrets, a durable database and uploads, HTTPS, operational monitoring and verified partner configuration.

07 / COMMERCIAL MODEL

Operations, billing and money flows

Business billing

Model	How it works	State / guard
Trial	First three completed pickups per business are free by default.	After trial, a plan is required to keep adding lots.
Pay per ride	Base pickup amount plus weight rates by route, using hub-confirmed weight at closure.	Invoices are created after the pickup is closed; an unpaid invoice blocks adding further lots.
Monthly	Monthly subscription with included kg quota; usage beyond quota is billed at configured per-kg rates.	Plan activation follows a paid invoice; subscription term is configured as 30 days.

Collector fare

Estimated fare formula: base + distance + extra stops + handled kilograms. Distance is estimated from straight-line coordinates multiplied by a configurable road factor. The fare is frozen at run close. All rate values are configurable assumptions, not validated market prices.

Payment behavior

The backend derives amounts from its own invoice, not a browser-provided amount. Razorpay checkout supports test/live mode gating and verifies the order/payment signature server-side. With no keys, a built-in simulator is available only outside production; it does not move real money. Live keys are refused unless explicitly enabled.

Commercial figures are placeholders

Default plan prices, driver fare parameters, facility rates, environmental factors and the pickup minimum are pilot assumptions/configuration. Confirm and approve pricing, taxes, contracts and payment operations before commercial launch.

08 / ASSURANCE

Security, privacy and readiness

Implemented control	What it contributes	Known limit / follow-up
Password and session	Scrypt password hashes; signed session cookie; password changes invalidate other sessions via session version.	No phone OTP/SMS verification today. Configure a strong production SECRET_KEY and HTTPS-only cookies.
Rate limits	Limits login, signup, reset and payment-related attempts.	In-memory only; use shared Redis or equivalent when scaling to multiple app processes.
Authorization and data scope	Role dependencies; business pickup views limit data to that business; invoice lookups enforce ownership.	Continue reviewing new endpoints for tenant-level access boundaries.
Request and browser controls	Same-origin write guard; CSP and common security headers; API no-store response cache policy.	Production deployment needs a trusted HTTPS proxy and careful forwarded-header configuration.
Certificate documents	Admin-only upload; signed-in viewing; content-signature checks; PDF/PNG/JPEG; 5 MB limit.	Uploaded certificate presence does not prove authenticity. Verify facility licenses against authoritative sources.
Record verification	Recomputes SHA-256 payload hash and previous-record link.	Detects edits within the application chain; does not make the record an official legal or regulatory certificate.

Pre-launch checklist

- Implement phone OTP or another verified identity flow; configure production reset email delivery.
- Have legal counsel review the Terms and Privacy copy; validate consent and retention practices.
- Onboard real facilities and independently verify certificates and service scope.
- Add schema migration management, backup/restore tests for database and uploads, monitoring and incident response.
- Replace placeholder environmental factors and pricing; validate route classification with domain/legal experts.
- Deploy behind HTTPS and test camera/PWA behavior on target mobile browsers; test network and firewall access for local demos.

09 / BUILD NOTES

Implementation approach and current scope

The implementation separates domain decisions from screens: the tier engine determines eligibility; facility matching selects an appropriate destination; dispatch groups and assigns runs; billing creates invoices from confirmed operational data; the record service seals the closeout payload. The JavaScript screens call the same-origin API and render role-specific workflows.

Notable product/data evolution

- Produce options use broad categories (Vegetables and Fruits) rather than one-off Tomato, Banana and Leafy greens choices.
- Compatibility aliases map historic item keys such as tomato, banana, greens and bread to their new category keys, protecting existing records/lots from a catalog rename.
- User-visible routing is labeled A-D while the backend uses stable internal route codes; this decouples presentation labels from stored route data.
- The former rescue route is retired: fresh, good-condition in-date food is not accepted by this paid disposal workflow.

Testing evidence

The repository test suite covers routing rules, authentication and API flows, pickup/record generation, dispatch capacity and assignment, facility certificate documents, pricing plans and payment-signature checks. At the time this overview was prepared, the full suite completed with 47 passing tests. This is software test evidence, not a production certification or real-world pilot result.

Current implementation status

Feature descriptions in this PDF reflect the codebase inspected on 26 October 2026. Live partner availability, operator performance, regulatory compliance, accurate environmental impact and production service levels have not been established by the test suite.

10 / PRESENTATION HANDOFF

Suggested PPT structure

Slide	Suggested title	Key message / visual
1	FoodLoop: safe disposal, visible proof	Problem statement and one-line proposition; use app cover visual.
2	The problem and opportunity	Operational waste routing challenge; distinguish disposal from food rescue.
3	Who uses FoodLoop?	Business, collector and platform admin persona cards.
4	How a lot gets routed	Simple decision tree: category + condition + expiry -> route.
5	From lot to planned pickup	Workflow swimlane: add, classify, match, pool, plan.
6	Dispatch and proof of disposal	Collector assignment -> schedule -> collection -> dual weights -> record/QR.
7	What each role can do	Feature matrix or annotated screens.
8	Technology and system design	PWA -> FastAPI -> services -> SQL DB; note support libraries and deployment.
9	Business model and controls	Trial, per-ride/monthly billing and test/live payment boundary.
10	Impact, caveats and next steps	Only validated metrics; list pilot assumptions, regulatory/legal and readiness work.

Recommended presentation discipline

Use the workflow and role capabilities as the product story. Mark pricing, driver fares and environmental figures as assumptions until validated. Say 'tamper-evident hash-chained record' rather than 'blockchain certificate'. Say 'certification information recorded by the platform' rather than claiming FoodLoop itself certifies facilities.

Source anchors in the repository

Core implementation references: app/tiers.py (classification and category aliases); app/matching.py (facility match and run building); app/dispatch.py (collector assignment and fare); app/records.py (record hash and PDF); app/billing.py and app/payments.py (plans/invoices/gateway); app/routes/ (REST workflows); static/js/ (role screens); Dockerfile and docker-compose.yml (deployment).

END OF PROJECT OVERVIEW